TaskFlow Backend Internship Project

1. Project Overview

TaskFlow is a backend-only task and project management application.

The purpose of this project is to give practical experience building a real-world backend using modern Python technologies and good software engineering practices.

The application will allow users to:

- Create and manage projects
- Add members to projects
- Create and manage tasks
- Assign tasks to users
- Add comments to tasks
- Track task activity
- Authenticate and authorize users

The project is intentionally smaller than a full enterprise application, but it should still cover the important concepts required to build a production-quality backend.

The backend will be built using FastAPI, SQLAlchemy, PostgreSQL, Alembic, Pydantic, and uv.

2. Technology Stack

The following technologies are required:

- Python 3.12+
- FastAPI
- Pydantic
- SQLAlchemy 2.x
- PostgreSQL
- Alembic
- uv
- Pytest
- HTTPX
- Docker
- JWT authentication

Each technology should be used for its intended purpose. You should be able to explain why each technology is being used.

3. Main Learning Objectives

By completing this project, you should gain practical experience with:

- Modern Python development
- Object-oriented programming
- Type hints
- FastAPI
- Pydantic
- SQLAlchemy
- PostgreSQL
- Alembic migrations
- REST API design
- Authentication
- Authorization
- Database relationships
- Transactions
- Pagination
- Filtering
- Testing
- Docker
- Dependency injection
- Clean backend architecture

The goal is not simply to make the API work. You should understand how every major part of the application works and why it is designed that way.

4. Object-Oriented Programming Requirement

The project must be developed using object-oriented programming principles.

Classes should be used where they provide a clear responsibility.

You are expected to understand and apply:

- Classes and objects
- Encapsulation

- Abstraction
- Composition
- Inheritance where appropriate
- Interfaces or abstract base classes where useful
- Dependency injection
- Separation of responsibilities

Do not create classes just to make the project look object-oriented.

The goal is to create a clean design where every major class has a clear purpose.

A typical request should follow a structure similar to:

HTTP Request

|

FastAPI Router

|

Service

|

Repository

|

SQLAlchemy

|

PostgreSQL

5. Python Type Hints

Type hints are mandatory throughout the project.

Use type hints for:

- Function parameters
- Function return values
- Classes
- Service methods

- Repository methods
- FastAPI dependencies
- Pydantic models
- SQLAlchemy-related code

Example:

```
async def get_task(
    task_id: int,
    db: AsyncSession,
) -> Task | None:
    ...
```

Avoid writing untyped functions when the type can be clearly defined.

Use modern Python typing syntax where appropriate, such as:

```
str | None
list[Task]
dict[str, str]
```

The code should be easy to understand with the help of its type annotations.

6. Project and Dependency Management with uv

The project must use uv as the Python project and dependency manager.

The project should contain:

```
pyproject.toml
uv.lock
```

Dependencies should be managed through uv.

Examples:

```
uv init
```

uv add fastapi

uv add sqlalchemy

uv add alembic

uv add pydantic

uv add psycopg

Development dependencies should also be managed through uv.

For example:

uv add --dev pytest

uv add --dev httpx

Do not use pip and requirements.txt as the primary dependency management system.

7. Application Architecture

The application should have a clear separation of responsibilities.

A recommended structure is:

app/

|

|-- main.py

|

|-- core/

| |-- config.py

| |-- database.py

| |-- security.py

|

|-- models/

```
|   |-- user.py
|   |-- project.py
|   |-- task.py
|   |-- comment.py
|   |-- activity.py
|
|-- schemas/
|   |-- user.py
|   |-- project.py
|   |-- task.py
|   |-- comment.py
|
|-- repositories/
|   |-- user.py
|   |-- project.py
|   |-- task.py
|   |-- comment.py
|
|-- services/
|   |-- auth.py
|   |-- project.py
|   |-- task.py
|   |-- comment.py
|   |-- activity.py
```

```
|
|-- permissions/
|   |-- project.py
|   |-- task.py
|
|-- dependencies/
|   |-- database.py
|   |-- authentication.py
|   |-- services.py
|
|-- api/
|   |-- v1/
|       |-- auth.py
|       |-- users.py
|       |-- projects.py
|       |-- tasks.py
|       |-- comments.py
|
|-- tests/
    |-- unit/
    |-- integration/


alembic/
|-- versions/
```

This is a recommended structure. You may change it if they have a valid architectural reason.

8. API Layer

FastAPI routers should be responsible mainly for HTTP-related concerns.

The API layer should handle:

- HTTP requests
- Request validation
- Authentication dependencies
- Calling services
- Response schemas
- HTTP status codes

Business logic should not be placed directly inside large route functions.

For example, a route should not contain all of the logic for checking project membership, creating a task, updating the database, and creating an activity record.

That responsibility should belong to the appropriate service.

9. Service Layer

Business logic should be implemented using service classes.

For example:

class TaskService:

```
    def __init__(
        self,
        task_repository: TaskRepository,
    ) -> None:
        self.task_repository = task_repository
```

```
    async def create_task(
        self,
        data: TaskCreate,
        user_id: int,
    ) -> Task:
        ...
```

A service may be responsible for:

- Checking business rules
- Checking permissions
- Validating relationships
- Coordinating multiple repositories
- Managing application workflows
- Creating activity records
- Managing transactions where appropriate

The service should not be responsible for HTTP-specific details.

10. Repository Layer

Database access should be separated from business logic.

For example:

```
class TaskRepository:


    def __init__(
        self,
        session: AsyncSession,
    ) -> None:
        self.session = session
```

```
    async def get_by_id(
        self,
        task_id: int,
    ) -> Task | None:
        ...


    async def create(
        self,
        task: Task,
    ) -> Task:
        ...


    async def delete(
        self,
        task: Task,
    ) -> None:
        ...
```

Repositories should be responsible for:

- SQLAlchemy queries
- Retrieving database records
- Creating records
- Updating records
- Deleting records

Repositories should not contain business decisions such as whether a particular user is allowed to delete a task.

That belongs in the service or authorization layer.

11. Repository Abstraction

Where it provides a real benefit, repository interfaces or abstract base classes may be used.

For example:

```
from abc import ABC, abstractmethod




class TaskRepositoryInterface(ABC):


    @abstractmethod

    async def get_by_id(

        self,

        task_id: int,

    ) -> Task | None:

        ...
```

A SQLAlchemy implementation can then provide the actual database behavior.

This is intended to teach:

- Abstraction
- Loose coupling
- Dependency inversion
- Testability

Do not create an interface for every class just for the sake of following a pattern.

12. Pydantic Requirements

Pydantic must be used for API request and response schemas.

SQLAlchemy models should not be used directly as the API contract.

For example:

```
class TaskCreate(BaseModel):
    title: str
    description: str | None = None
    priority: TaskPriority
    due_date: date | None = None
```

And a separate response schema:

```
class TaskResponse(BaseModel):
    id: int
    title: str
    status: TaskStatus
    priority: TaskPriority


    model_config = ConfigDict(
        from_attributes=True,
    )
```

Pydantic should be used for:

- Request validation
- Response serialization
- Nested response models
- Enum validation
- Optional fields
- API data contracts

You should understand the difference between a Pydantic schema and a SQLAlchemy model.

13. Database

PostgreSQL must be used as the database.

SQLAlchemy 2.x must be used for database access.

The main database entities should include:

users

projects

project_members

tasks

comments

activity_logs

You are responsible for designing the relationships between these entities.

The database should make appropriate use of:

- Primary keys
- Foreign keys
- Unique constraints
- Indexes
- Relationships
- Cascades
- Timestamps

You should be able to explain their database design.

14. SQLAlchemy Requirements

During the project, you should demonstrate knowledge of:

- Declarative models
- SQLAlchemy sessions
- Async database access
- select queries
- Filtering
- Joins
- Relationships
- Transactions
- Relationship loading
- Constraints
- Indexes

You should also understand common ORM problems such as N+1 queries and unnecessary database queries.

Do not assume that using an ORM automatically makes database queries efficient.

15. Alembic Requirements

All database schema changes must be managed using Alembic.

You should create migrations for:

- Initial tables
- New columns
- Constraints
- Indexes
- Relationship changes

They should be able to use commands such as:

alembic revision --autogenerate

alembic upgrade head

alembic downgrade -1

alembic history

alembic current

Do not manually modify the database schema without creating the corresponding migration.

Autogenerated migrations should always be reviewed before being applied.

16. User Authentication

The application must support:

- User registration
- User login
- User logout
- Viewing the current user
- Updating the current user

Use JWT-based authentication.

Passwords must never be stored as plain text.

Password-related functionality may be separated into a service such as:

class PasswordService:

```
    def hash(self, password: str) -> str:
        ...

    def verify(
        self,
        password: str,
        password_hash: str,
    ) -> bool:
        ...
```

Token functionality may similarly be separated into a dedicated service.

The exact implementation is part of the assignment.

17. Authorization

Authentication answers:

"Who are you?"

Authorization answers:

"Are you allowed to perform this action?"

TaskFlow must implement authorization.

For example:

- Project owners can update and delete projects.
- Project members can view project information.
- Project members can create tasks.
- Only authorized users can modify tasks.
- Users can edit their own comments.
- Users cannot access projects they do not belong to.

Authorization logic should be reusable rather than duplicated across multiple routes.

18. Projects

Authenticated users should be able to:

- Create projects
- View their projects
- View a specific project
- Update projects
- Delete projects
- Add members
- Remove members
- View project members

A project should contain at least:

- ID
- Name
- Description
- Owner
- Created timestamp
- Updated timestamp

A user should not be allowed to access projects they are not authorized to access.

19. Project Members

A project can contain multiple users.

The system should support:

- Adding project members
- Listing project members
- Removing project members

The same user must not be added to the same project more than once.

This should be enforced at the database level as well as handled by application logic.

This feature is intended to provide experience with many-to-many relationships.

20. Tasks

Tasks are the main feature of TaskFlow.

Each task should contain:

- ID
- Title
- Description
- Status
- Priority
- Project
- Creator
- Assignee
- Due date
- Created timestamp
- Updated timestamp

Task statuses:

TODO

IN_PROGRESS

DONE

Task priorities:

LOW

MEDIUM

HIGH

Users should be able to:

- Create tasks
- View tasks
- Update tasks

- Delete tasks
- Assign tasks
- Change status
- Change priority

Only users with appropriate access to the project should be able to interact with its tasks.

21. Task Filtering and Pagination

The task listing endpoint must support pagination.

For example:

GET /api/v1/projects/{project_id}/tasks?page=1&page_size=20

It should also support filtering by:

- Status
- Priority
- Assignee

Examples:

GET /api/v1/projects/{project_id}/tasks?status=TODO

GET /api/v1/projects/{project_id}/tasks?priority=HIGH

GET /api/v1/projects/{project_id}/tasks?status=IN_PROGRESS&priority=HIGH

Filtering and pagination should be performed through database queries.

Do not retrieve the entire dataset into Python and then filter it.

22. Comments

Users should be able to add comments to tasks.

A comment should contain:

- ID

- Task
- Author
- Content
- Created timestamp
- Updated timestamp

Users should be able to:

- Create comments
- View comments
- Edit their own comments
- Delete their own comments

Users should not be able to edit another user's comments unless the authorization rules explicitly allow it.

23. Activity History

The system should track important actions performed on tasks.

Examples include:

- Task created
- Task updated
- Task assigned
- Task status changed
- Task priority changed
- Comment added

An activity record should contain information such as:

- User who performed the action
- Task
- Action
- Relevant details
- Timestamp

For example:

John changed "Fix Login API"

from TODO to IN_PROGRESS

The database design for activity history is part of the assignment.

24. Transactions

You must understand and correctly use database transactions.

For example, adding a project member may require:

1. Verify that the user exists.
2. Verify that the project exists.
3. Check whether the user is already a member.
4. Create the membership.
5. Commit the transaction.

If the operation fails, the database should not be left in an inconsistent state.

You should understand:

- Commit
- Rollback
- Flush
- Refresh
- Integrity errors
- Transaction boundaries

25. Error Handling

The API should return appropriate HTTP status codes.

You should understand and correctly use:

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Unprocessable Entity

500 Internal Server Error

Examples of errors that should be handled include:

- Invalid authentication
- Unauthorized access
- Resource not found
- Duplicate project membership
- Invalid request data
- Database constraint violations

Internal database errors should not be exposed directly to API clients.

26. API Versioning

The API should be versioned.

All endpoints should use:

/api/v1/

For example:

/api/v1/auth/login

/api/v1/projects

/api/v1/tasks

FastAPI's automatic API documentation should be kept clear and useful.

Endpoints should have appropriate:

- Request schemas
- Response schemas
- Descriptions
- Status codes

27. Required API Endpoints

The final API should provide functionality similar to the following.

Authentication:

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/logout

Users:

GET /api/v1/users/me

PATCH /api/v1/users/me

Projects:

POST /api/v1/projects

GET /api/v1/projects

GET /api/v1/projects/{id}

PATCH /api/v1/projects/{id}

DELETE /api/v1/projects/{id}

Project members:

POST /api/v1/projects/{id}/members

GET /api/v1/projects/{id}/members

DELETE /api/v1/projects/{id}/members/{user_id}

Tasks:

POST /api/v1/projects/{id}/tasks

GET /api/v1/projects/{id}/tasks

GET /api/v1/tasks/{id}

PATCH /api/v1/tasks/{id}

DELETE /api/v1/tasks/{id}

Comments:

POST /api/v1/tasks/{id}/comments

GET /api/v1/tasks/{id}/comments

PATCH /api/v1/comments/{id}

DELETE /api/v1/comments/{id}

Activity:

GET /api/v1/tasks/{id}/activity

Additional endpoints may be added if they improve the project.

28. Testing

Testing is a required part of the project.

Use Pytest.

At minimum, write tests for:

Authentication:

- User registration
- Successful login
- Invalid credentials
- Protected endpoints

Projects:

- Create project
- Retrieve project
- Update project
- Delete project
- Unauthorized access

Project members:

- Add member
- Remove member
- Duplicate member

Tasks:

- Create task
- Update task
- Delete task
- Assign task
- Change status
- Change priority

- Filtering
- Pagination

Comments:

- Create comment
- Update own comment
- Attempt to update another user's comment
- Delete comment

Tests should cover both successful operations and failure cases.

29. Object-Oriented Testing

The service and repository layers should be designed so that business logic can be tested independently.

For example, a TaskService should be testable without always requiring the complete API stack.

This will provide practical experience with:

- Unit testing
- Mocking
- Dependency injection
- Separation of concerns
- Testable architecture

The project should contain both unit tests and integration/API tests.

30. Docker

The application should run using Docker.

At minimum, Docker Compose should provide:

FastAPI

PostgreSQL

The project should be easy for another developer to run locally.

The README should contain clear instructions for starting the application, running migrations, and running tests.

31. Coding Standards

The code should be:

- Readable
- Properly typed
- Modular
- Testable
- Consistent
- Easy to understand

Avoid:

- Large route functions
- Duplicate code
- Unnecessary classes
- Unnecessary inheritance
- Unnecessary abstractions
- Poor variable names
- Untyped functions
- Business logic inside routers
- Direct database access scattered throughout the application

Prefer composition over deep inheritance hierarchies.

The objective is clean architecture, not maximum complexity.